

Pontifícia Universidade Católica de Campinas
Escola Politécnica

LAYOUT COM CSS

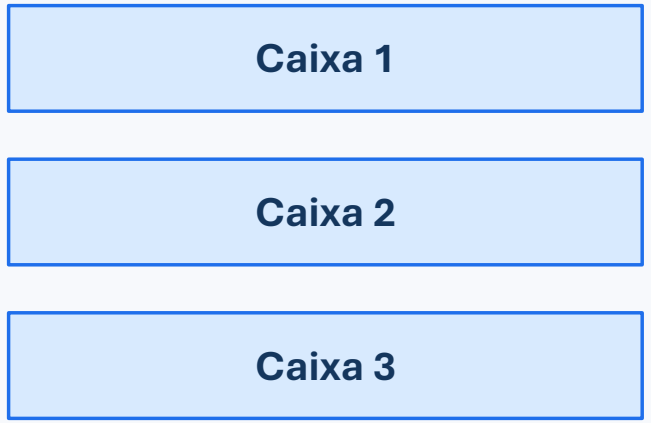
INTRODUÇÃO AS TECNOLOGIAS PARA WEB

José Guilherme Picolo
E-mail: jose.picolo@puc-campinas.edu.br

Pergunta inicial: por que as divs empilham?

```
<div class="caixa">Caixa 1</div>
<div class="caixa">Caixa 2</div>
<div class="caixa">Caixa 3</div>
```

Resultado padrão



Resposta:
porque a div, por padrão, possui comportamento de bloco: ela inicia em uma nova linha.

Novo tema:
a propriedade display controla esse comportamento.

display: o comportamento do elemento

```
seletor {
    display: valor;
}
```

Valores que veremos hoje

block **inline** **inline-block** **flex**

Ideia:
display não muda o conteúdo; muda a forma como o elemento participa do layout da página.

display: block

Conceito:
elementos block começam em uma nova linha e aceitam largura e altura normalmente.

```
.exemplo-block {
    display: block;
    width: 200px;
    height: 40px;
}
```

Comportamento visual

Elemento 1

Elemento 2

Elemento 3

Exemplos comuns: div, p, h1, section

display: inline

Conceito:
elementos inline permanecem na mesma linha e ocupam principalmente o espaço do conteúdo.

```
.exemplo-inline {
    display: inline;
    width: 200px;
    height: 100px;
}
```

Comportamento visual



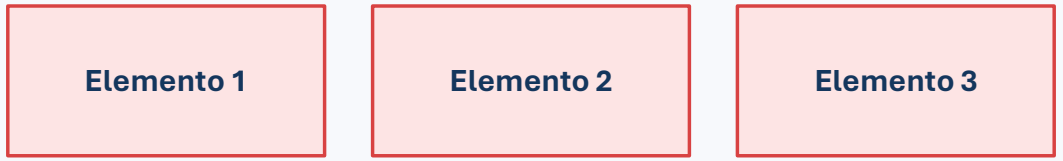
Importante:
width e height não se comportam como em block.

display: inline-block

Conceito:
inline-block combina o melhor dos dois mundos: fica na mesma linha, mas aceita width e height.

```
.exemplo-inline-block {
    display: inline-block;
    width: 150px;
    height: 60px;
}
```

Comportamento visual



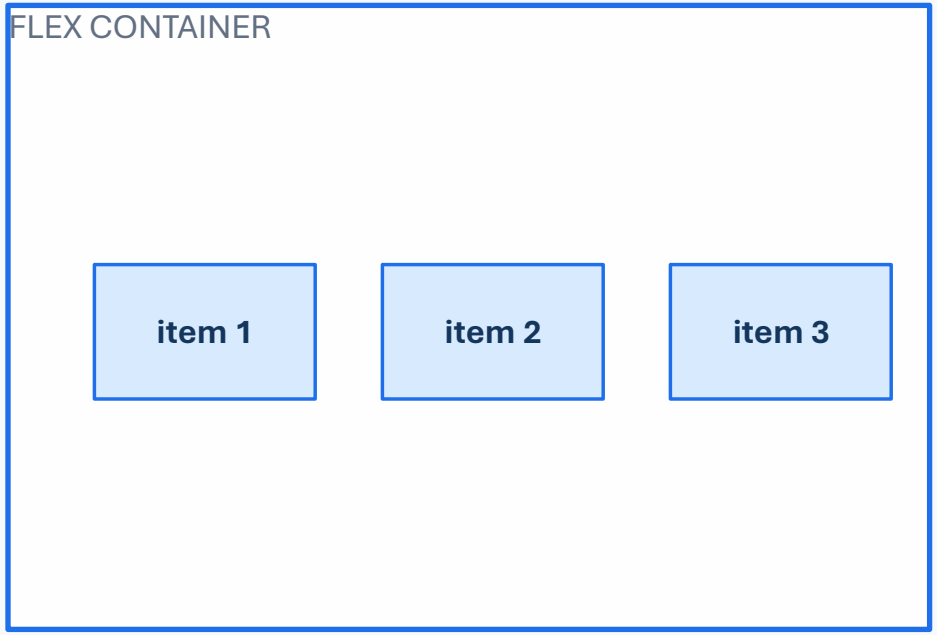
Funciona em casos simples, mas layouts maiores exigem uma ferramenta mais robusta.

Flexbox: contêiner e itens

Definição:
Flexbox é um sistema de layout para organizar, alinhar e distribuir elementos dentro de um contêiner.

```
.container {
    display: flex;
}
```

Regra de ouro:
display: flex vai no elemento PAI.

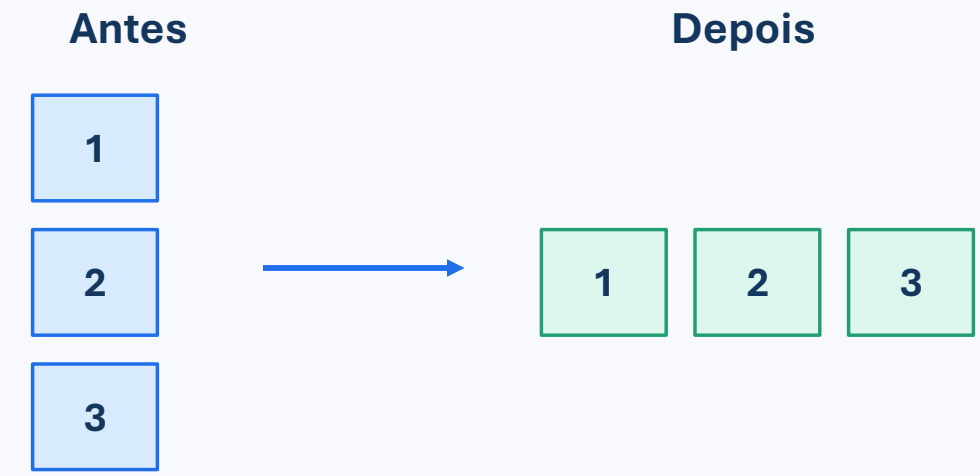


filhos diretos = flex items

Primeiro exemplo com display: flex

```
<div class="container-flex">
    <div class="item">1</div>
    <div class="item">2</div>
    <div class="item">3</div>
</div>
```

```
.container-flex {
    display: flex;
}
```



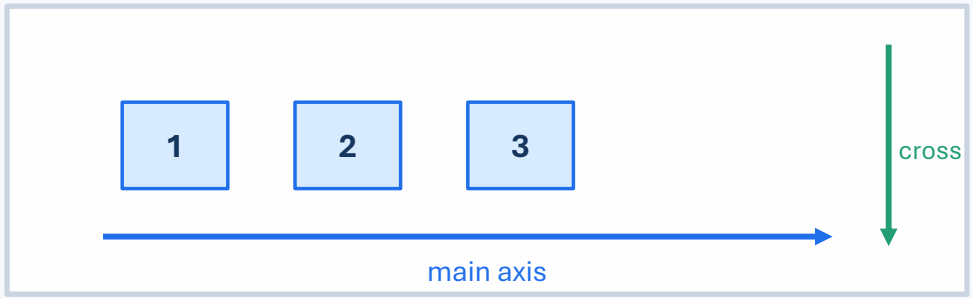
Efeito inicial:
os itens deixam de empilhar e passam a ficar no mesmo eixo.

Os dois eixos do Flexbox

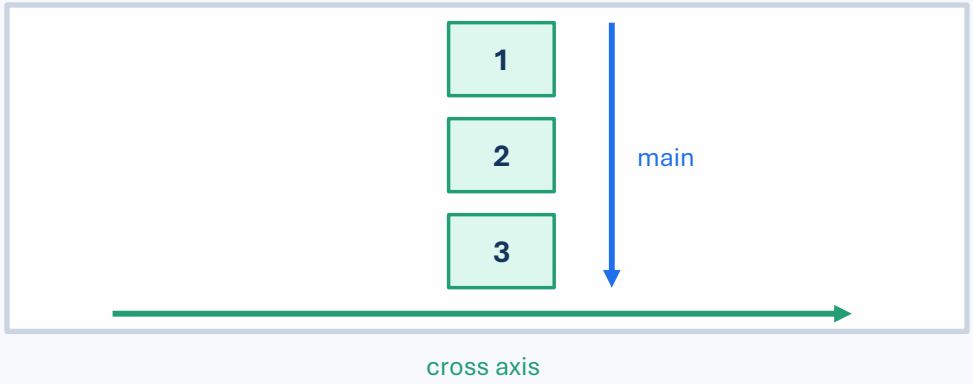
Ideia essencial:
Flexbox sempre trabalha com eixo principal e eixo transversal. Eles dependem do flex-direction.

Evite decorar:
justify-content não é “sempre horizontal”; align-items não é “sempre vertical”. Depende dos eixos.

flex-direction: row

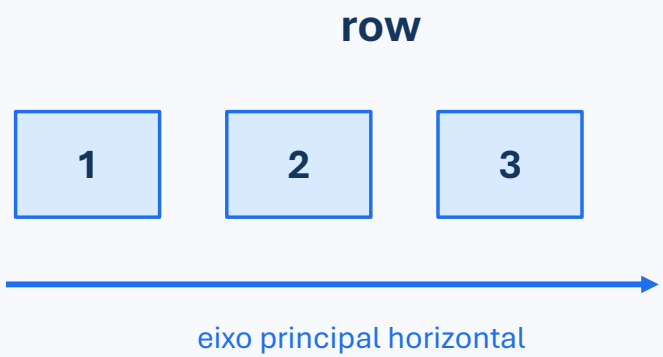


flex-direction: column

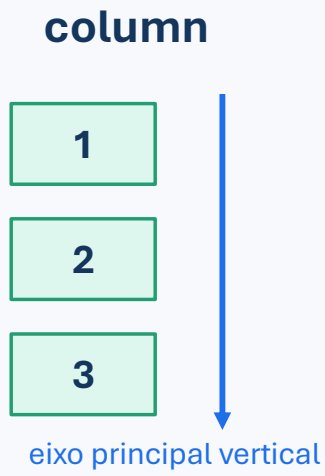


flex-direction: direção do eixo principal

```
.container {
    display: flex;
    flex-direction: row;
}
```




```
.container {
    display: flex;
    flex-direction: column;
}
```



justify-content: distribuição no eixo principal

Definição:
organiza os itens ao longo do eixo principal.

```
.container {
    display: flex;
    justify-content: center;
}
```

flex-start



center



space-between

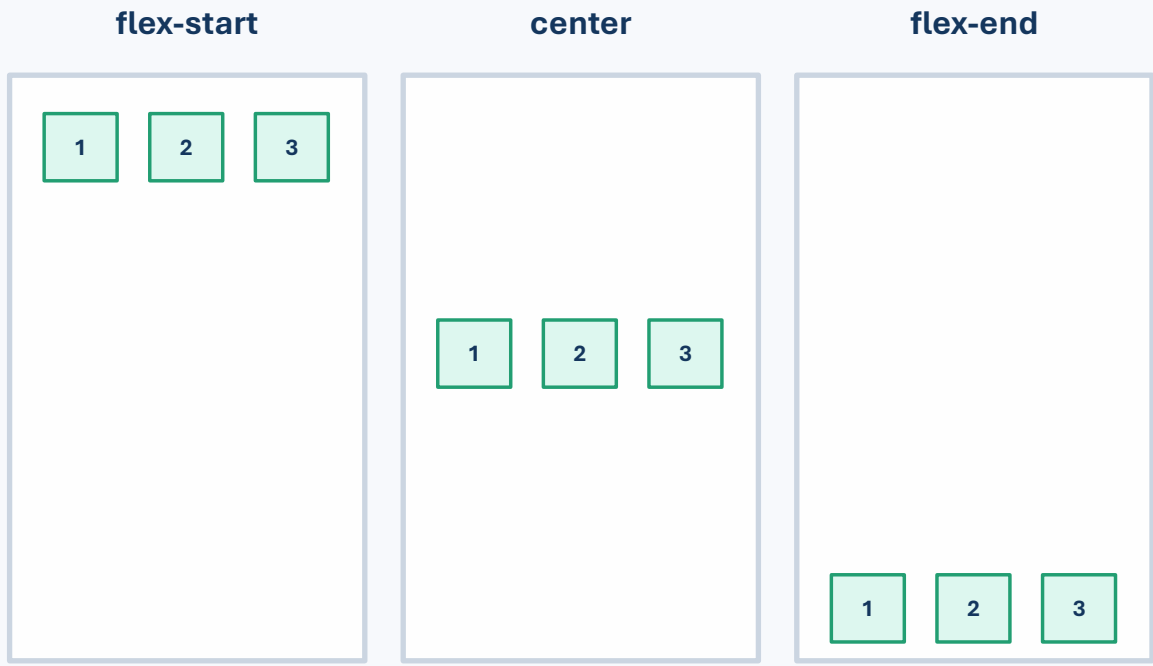


Atenção:
para perceber justify-content, precisa existir espaço sobrando no contêiner.

align-items: alinhamento no eixo transversal

Definição:
organiza os itens ao longo do eixo transversal.

```
.container {
    display: flex;
    height: 200px;
    align-items: center;
}
```

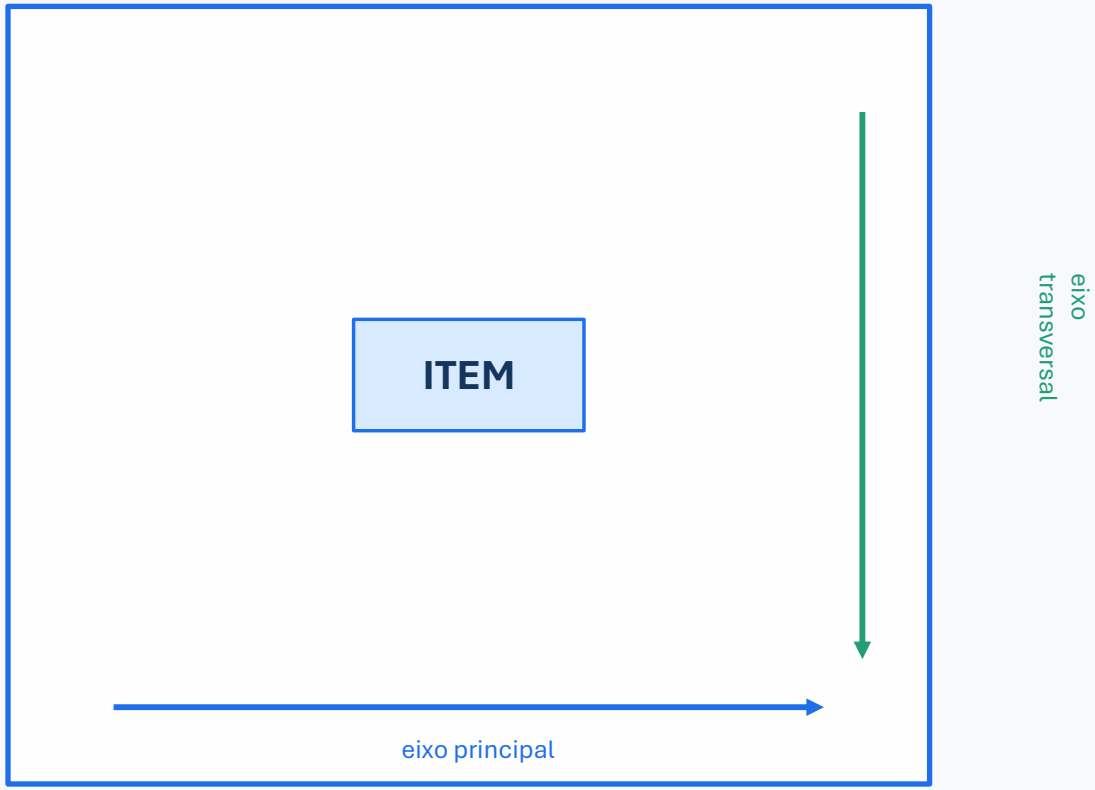


Dica:
coloque height no contêiner para o alinhamento vertical ficar visível.

Centralização em dois eixos

```
.container {
    display: flex;
    justify-content: center;
    align-items: center;
}
```

Leitura correta:
justify-content centraliza no eixo principal; align-items centraliza no eixo transversal.



gap: espaço entre os itens

Conceito:
gap define o espaçamento entre os filhos de um contêiner flex.

```
.container {
    display: flex;
    gap: 30px;
}
```

Sem gap



Com gap: 30px



Comparação:
margin fica nos itens; gap fica no contêiner.

flex-wrap: quebrar linha quando faltar espaço

Conceito:
por padrão, o Flexbox tenta manter tudo na mesma linha. Com wrap, os itens podem quebrar.

```
.container {
    display: flex;
    flex-wrap: wrap;
    gap: 20px;
}
```

nowrap



wrap



Ponte para responsividade: o layout começa a se adaptar ao espaço disponível.

Exemplo completo: catálogo de produtos

```css
.produtos {
    display: flex;
    justify-content: center;
    gap: 20px;
    flex-wrap: wrap;
}

.produto {
    width: 220px;
}
```

Resultado esperado

